StockSync: Inventory Traceability and Recall Management System

DBMS Level 3 Project Report — Working Draft

Team members:

Rekha Dhorigol, PES1UG24CS370

Reet Porwal, PES1UG24CS369

Section: G

1. Abstract

StockSync is a web-based inventory traceability system for products, supplier batches, warehouse stock, reservations, transfers, shipments, quality defects and product recalls. It connects daily inventory operations with traceability and recovery workflows instead of providing only basic CRUD. MySQL stores structured operational data and enforces relational integrity. MongoDB stores event documents synchronized from a MySQL transactional outbox. Views support reporting, stored procedures perform multi-step stock/recall operations, and triggers publish inventory changes into the outbox. The Next.js application supports ADMIN, INVENTORY_STAFF and QUALITY_STAFF roles.

2. Problem Statement and Objectives

Inventory traceability becomes difficult when stock is split across warehouses, reserved, transferred, shipped, or quarantined after a quality issue. A recall requires identifying affected batches, their stock locations and downstream recipients while ensuring unsafe stock is not available.

- Track products and supplier batches through warehouses and shipments.
- Track on-hand, reserved, quarantined and derived available quantities.
- Manage reservations, transfers, shipment progress, discrepancies, defects and recalls.
- Enforce data integrity using keys, checks, indexes and transactional procedures.
- Separate access by user role.
- Synchronize operational events from MySQL to MongoDB using an outbox workflow.

3. Functional Requirements

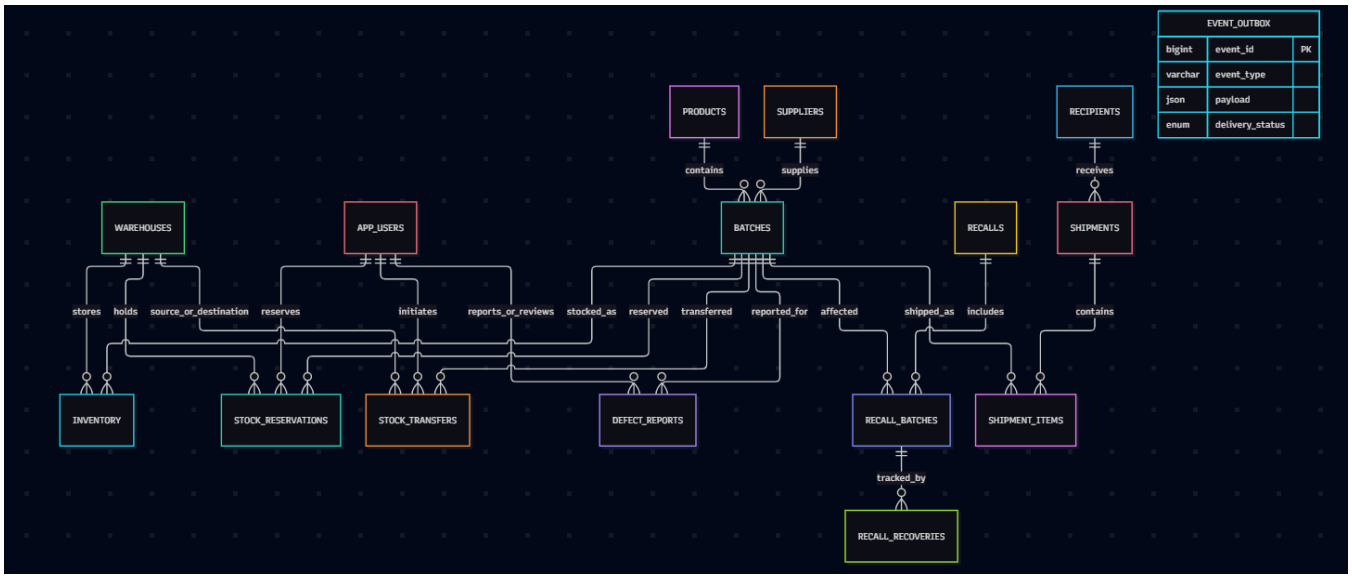
Requirement	Description
Authentication	Role-aware sign-in for administrators, inventory staff and quality staff.
Inventory	View stock by product, batch and warehouse; account for reservations and quarantine.
Reservations/transfers	Reserve or cancel stock and transfer batches between warehouses.
Shipments	Track shipment lifecycle and dispatched/received quantities; identify discrepancies.
Quality/recalls	Record defects, review reports, initiate recalls, quarantine stock and record recovery.
Event sync	Write inventory events to a MySQL outbox and synchronize event documents to MongoDB.

4. Technology and Database Choice

Frontend/application: Next.js, React and TypeScript. Relational database: MySQL/InnoDB. Document database: MongoDB. Supporting libraries include mysql2, mongodb, Zod, bcryptjs and jose. MySQL is appropriate for related records, joins, constraints and transactions. MongoDB stores event documents with event-type-specific payloads. The implementation uses a document database; it does not claim vector search.

5. ER Design and Relationships

- products 1—many batches; suppliers 1—many batches.
- batches 1—many inventory rows; warehouses 1—many inventory rows. A unique (batch_id, warehouse_id) constraint prevents duplicate stock rows.
- batches 1—many defect_reports; app_users can be reporters/reviewers.
- recalls and batches connect through recall_batches; recall_recoveries records actions against recall/batch/warehouse.
- recipients 1—many shipments; shipments 1—many shipment_items; each shipment item references a batch.
- stock_reservations reference a batch, warehouse and optional user; stock_transfers reference a batch, source/destination warehouses and optional user.
- event_outbox identifies each event by event_id and its aggregate type/id; events are synchronized to MongoDB.



(Done via mermaid)

6. Data Dictionary — Core Tables

Table	Important attributes/constraints	Purpose
app_users	user_id PK; full_name; email UNIQUE; password_hash; role ENUM; is_active; created_at	Accounts and roles.
products	product_id PK; SKU and product attributes as defined in SQL	Product master.
suppliers	supplier_id PK; supplier_name; contact_name; email; phone; address; status	Supplier master.
batches	batch_id PK; batch_code UNIQUE; product_id FK; supplier_id FK; manufacture/expiry dates; unit_cost; status	Batch traceability.
warehouses	warehouse_id PK; warehouse_code UNIQUE; name; address; city; capacity; status	Warehouse locations.
inventory	inventory_id PK; batch_id FK;	Stock balance.

	warehouse_id FK; quantity_on_hand/reserved/quarantined; generated available_quantity; UNIQUE(batch_id,warehouse_id)	
stock_reservations	reservation_id PK; batch_id/warehouse_id/user FKs; quantity; status; expires_at	Reservation lifecycle.
stock_transfers	transfer_id PK; batch/source/destination/user FKs; quantity; status; notes	Warehouse movement.
recipients	recipient_id PK plus organization/contact fields in SQL	Shipment destinations/recall targets.
shipments	shipment_id PK; shipment_code UNIQUE; recipient/warehouse/user FKs; status; dispatch/delivery times	Shipment lifecycle.
shipment_items	item key; shipment_id FK; batch_id FK; quantity_dispatched; quantity_received; discrepancy notes	Batch quantities per shipment.
defect_reports	defect_id PK; defect_code UNIQUE; batch FK; defect type; description; affected quantity; severity/status; reporter/reviewer FKs	Quality reporting.
recalls	recall_id PK and recall metadata/status fields in SQL	Recall cases.
recall_batches	recall/batch association and affected/quarantine quantities	Batches affected by recalls.
recall_recoveries	recall/batch/warehouse references; quantity; action_type; recorded_by; notes	Quarantine/recovery records.
event_outbox	event_id PK; event_type; aggregate_type/id; JSON payload; delivery_status; attempts; timestamps/error	Reliable event handoff.

7. Integrity, Indexing and Advanced SQL

- Primary/foreign keys connect products, batches, suppliers, warehouses, users, inventory, shipments and recalls.
- Unique constraints cover email, batch code, warehouse code, shipment code, defect code and batch/warehouse inventory.
- CHECK constraints validate non-negative unit cost, batch date order, reservation/transfer quantities, distinct transfer warehouses, inventory quantity bounds and shipment date order.
- inventory.available_quantity is a stored generated value: on-hand − reserved − quarantined.
- Views: vw_inventory_overview, vw_low_stock, vw_recall_recipient_trace, vw_shipment_discrepancies.
- Stored procedures: sp_reserve_stock, sp_cancel_reservation, sp_record_quarantine and the stock-transfer procedure.
- Triggers: trg_inventory_insert_outbox and trg_inventory_update_outbox create outbox events automatically.
- Transactions, SELECT ... FOR UPDATE, COMMIT/ROLLBACK and error handlers protect multi-step operations.
- Indexes support stock availability, batch expiry/status, outbox delivery, reservations and shipment status/date.

8. MongoDB Model and Architecture

MongoDB database: inventory_events; collection: events. Each document represents an event and carries event-type-specific payload and aggregate metadata. Inventory triggers write to the MySQL outbox; the synchronization API processes pending rows, writes documents to MongoDB and updates delivery status in MySQL. This reduces the risk of losing an event when an inventory transaction commits. This is an event-document pattern, not vector similarity search.

Architecture: Browser UI (Next.js/React) → Next.js API handlers → MySQL operational tables/views/procedures. The event-sync handler additionally writes event documents to MongoDB. Authentication uses bcrypt password hashing and jose-based sessions; Zod validates inputs. Configure credentials using environment variables and do not commit secrets.

9. Testing Evidence

Test	Recorded result (reconfirm before submission)
Lint/build	npm run lint and npm run build passed.
Reservation	Created and cancelled a reservation; reserved inventory returned to zero.
Shipment status	PREPARING → DISPATCHED → IN_TRANSIT succeeded; invalid DISPATCHED → CANCELLED returned HTTP 409.
Recall/quarantine	A quarantine recovery was recorded and a recovery row created.
SQL-to-MongoDB	Sync processed one pending event; processed=1, delivered=1, failed=0; SQL outbox status became DELIVERED.
Still required	Insert real screenshots of UI, SQL query/procedure results, MongoDB event document and tests.

10. Setup Notes, Limitations and Conclusion

- Use the repository README and .env.example; configure MySQL and MongoDB credentials locally.
- The SQL dump contains DROP TABLE statements and sample data. Import only into a fresh disposable development database, never a database whose data must be preserved.
- The dump has DEFINER clauses for root@localhost; adjust privileges/definers if import errors occur on another server.
- The dump disables foreign_key_checks during import; verify all expected tables, views, triggers and procedures after importing into a clean test database.
- Before submission, fill team/course details, render the ER diagram, add screenshots, and include representative JOIN, GROUP BY/HAVING, subquery, view/procedure and MongoDB query outputs.

Conclusion: StockSync combines normalized relational records with transactional stock/recall operations, database-enforced constraints, reporting views, stored procedures, triggers and a MongoDB event collection. Its focus is the traceable connection between stock movement, shipment discrepancies, quality defects and recall recovery. This is a working draft: verify exact column lists for all tables and add the required visual/test evidence before final export.